

Turnitin Check

RoadCare_Research_Paper

 Quick Submit Quick Submit Loyola University, Chicago (Sakai 12 Legacy API)

Document Details

Submission ID

trn:oid:::1:3537317493

Submission Date

Apr 14, 2026, 10:02 AM CDT

Download Date

Apr 14, 2026, 10:08 AM CDT

File Name

RoadCare_Research_Paper.docx

File Size

2.2 MB

7 Pages

5,898 Words

29,290 Characters

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



A Multi-Model Deep Learning Framework for Real-Time Road Damage Detection and Severity Classification

Dr. Lalband Neelu
Department of IT Malla Reddy
University
Hyderabad, India
drlalband.neelu@mallareddyuniversity.ac.in

Jelsinge Arun
Department of IT Malla Reddy
University
Hyderabad, India
jelsingearun.2004@gmail.com

Kalangi Lakshmi Sai
Chinmayee
Department of IT Malla Reddy
University
Hyderabad, India
chinmayee.kalangi@gmail.com

Kondarparthi Sandeep
Department of IT Malla Reddy
University
Hyderabad, India
sandeepkondarparthi717@gmail.com

D. Sai Sindhu
Department of IT Malla Reddy
University
Hyderabad, India
saisindhudongari@gmail.com

Abstract

Cities are having issues with potholes as roads are deteriorating. They influence the level of safety of driving among people as well as the level of money used in repairing the car. Manual checking, which is the common method of checking roads and involves people examining the roads to identify issues is not a limitless process. The results can vary depending on who is checking and it is difficult to do on a scale and may take a long time to discover and remedy issues.

We need a way to check roads that can work on a big scale and give us information in real-time. This essay describes a system which employs AI and citizen data to identify potholes and determine risks. To ensure that the system functions effectively, a combination of deep learning and computer vision is used. Pictures of the road are analyzed by computer vision. Checks of some features such as dark areas, edges and texture. A model that is used to learn the appearance of potholes with deep learning is referred to as YOLOv8.

Another method of storing information regarding the locations of potholes known as H3 indexing is also used in the system. This will assist in clustering potholes by location. Those with the highest priority are those to fix. The system is developed with FastAPI, which is a type of backend, MongoDB, and which is a type of database and React which is a type of frontend.

We evaluated the system on road picture dataset. It was more effective than the other techniques. Even in case of conditions, such as lighting and road texture, it was more realistic and sturdy. The system can assist in shortening the period required to check roads and make them fixed,

and allow citizens to engage in the management of city structures.

Keywords: *pothole detection; computer vision; deep learning; YOLOv8; clustering; H3 indexing; FastAPI; MongoDB; road damage assessment; civic technology; hybrid AI systems*

I. INTRODUCTION

Urban roads are really important for our transportation systems. They affect the economy how easily people can move around and public safety. One of the problems with urban roads is potholes. Potholes are holes in the road that form because of cars driving on the road water getting in and the weather. They can cause a lot of problems like damaging cars stopping traffic and making accidents more likely. Even though potholes are a problem finding and fixing them is not very efficient in many cities. This is because the usual way of checking the roads is not very good.

The usual way of checking the roads is for people to go out and look at the roads. This way of checking is not very good because it does not cover all the roads it does not happen often and the people looking at the roads might not always agree on how bad the problems are. This means that there is a delay between when a pothole forms and when it gets fixed. This delay makes the problem worse. Costs more money to fix. Also this way of checking the roads does not use the fact that many people are driving on the roads and could be helping to find potholes.

New computer technologies, like computer vision and deep learning can help find potholes. Some computer models, like YOLO are very good at finding objects, including potholes. However these models need a lot of data and computer power, which can be a problem. They also do not

always work well in lighting conditions or with different types of roads.

One can also look at pictures in different ways that can assist in locating potholes. These ways are not as powerful as the computer models but they can be more efficient and easier to understand. The problem is that we do not have a system that combines the things about the old ways and the new computer models.

Finding potholes is not the problem. We also need to organize the information about the potholes and decide which ones to fix first. The usual ways of doing this are not very good because they do not work well with the way cities are organized. We need a way to organize the information about potholes so that we can make good decisions about which ones to fix.

This paper is about a system that uses computer vision and citizen participation to find potholes and decide which ones to fix. The system uses a combination of new computer technologies to verify the location of potholes. It also uses a way of organizing the information about potholes called H3 hexagonal hierarchical spatial indexing. This system is a full-stack web application that allows citizens and city officials to work together to find and fix potholes.

The main contributions of this paper are A new way of verifying potholes that combines new computer technologies. A new way of organizing the information, about potholes using H3 hierarchical spatial indexing. A complete system that allows citizens and city officials to work together to find and fix potholes. The goal of this system is to find potholes quickly fix them more efficiently and make urban road management better.

II. LITERATURE REVIEW

Research on finding road damage with machines has changed a lot over time. There are a few ways to do this: using sensors looking at pictures with special computer programs and using old image processing techniques. All these ways have both advantages and disadvantages.

Sensors to detect potholes were used by people at one time. They would install gadgets in vehicles or cellphones to gauge the roughness of the road. If the road was really bumpy it might mean there was a pothole. This way is cheap and easy. It has some problems. It does not work well if the car is going fast or if the road is bumpy for some other reason. Also the people in charge of the roads cannot see the pothole for themselves so they do not know if it is really a problem.

Later computers got better at looking at pictures. This meant that people could use programs to find road damage in images. These programs like the ones called Convolutional Neural Networks are really good at finding potholes. Some programs like YOLO can even find potholes in time.. These programs need a lot of pictures to learn from and they do not work well if the pictures are not clear.

At the time people were also using old image processing techniques to find road damage. These techniques are fast and easy to use. They are not as good as the new computer

programs. They make a lot of mistakes. Do not work well in real-life situations.

Some people have tried to combine the techniques with the new computer programs. This means using the techniques to find possible potholes and then using the computer programs to make sure they are really potholes. This way works better than using one technique but it is still not perfect.

Road damage detection is not about finding potholes. It is also about figuring out where they are and how to fix them. Some people have used algorithms to group potholes together based on where they are.. These algorithms have some problems. They need to be run every time new data comes in and they do not always group the potholes in a way that makes sense.

There is a way of grouping potholes that uses hexagons. This way is better because it makes it easy to look at the potholes and figure out how to fix them.. It is not being used very much yet. There are also platforms that let people report road damage. These platforms are helpful. They have some problems. They need people to look at each report and figure out if it is real which takes a lot of time.

In general people have made a lot of progress in finding road damage and figuring out how to fix it.. There is still a lot of work to be done to make it all work together smoothly. That is what this project is trying to do: make a system that can find road damage figure out where it is and help people fix it all, in one place.

III. METHODOLOGY AND PROPOSED SYSTEM

A) System Overview

The RoadCare system is made up of three parts. It gets reports from citizens. It resorts to smartness to confirm pictures contained on these reports. It determines the risk areas using the location of the reports. RoadCare system may be compared to a building that has floors. A set of rules are used by the application layer and the database to communicate with one another. It is referred to as an API. Reports can be managed simultaneously without slowing down the RoadCare system.

Each report contains a picture, and the place, where the picture was shot and some additional information. When a report is forwarded to the RoadCare system it does things. It verifies to ensure that the report is good. It employs the speed to examine the image. It categorizes the report according to place. It records the report to have the opportunity to view it later. All this is done by the RoadCare system to ensure every report is attended to with the correct labels and stored to be reviewed and used to make decisions.

RoadCare system is report-based. It is able to work with things simultaneously. This is significant since it implies that the RoadCare system will not be slowed down whenever numerous individuals are utilizing it in submitting reports. The RoadCare system is able to manage reports without any issues.

B) Pothole Verification using AI.

The RoadCare system makes use of an application termed AIVerificationService to determine whether a given picture contains a pothole. This is a tool that analyzes the picture in five directions. It provides a score each time it needs to determine whether the picture actually has a pothole or not. This tool is used to check the pictures by the RoadCare system.

First it searches regions of the image. It transforms the image into black and white. It locates the sections extremely dark. In case it has many of these parts then it could be a pothole. The picture is checked with the RoadCare system.

Second it looks for edges in the picture. It makes the picture a little blurry. Then it finds the edges. If the edges are not smooth it could be a pothole. The RoadCare system checks the edges of the picture.

Third it looks at the texture of the picture. It divides the picture into parts. It looks at how different each part's from the others. When the components are highly dissimilar it may be a pothole. The system RoadCare examines the texture of the picture.

Fourth it looks at how light and dark the picture's. When there are many light and very dark areas in the picture it may be a pothole. RoadCare system analyses the dark areas on the picture.

Fifth it searches after shapes that may be potholes. It makes the picture a lot simpler. It discovers the forms in it. In case it adopts a form resembling a pothole it scores. The shapes in the picture are checked using the RoadCare system.

All these scores are summed up to obtain a score on the RoadCare system. When the final score is large enough the RoadCare system indicates that the picture contains a pothole. The score is determined through a formula that is provided in the RoadCare system.

The formula is: $C = 0.25 * S1 + 0.20 * S2 + 0.20 * S3 + 0.15 * S4 + 0.20 * S5$.

Where S1, S2, S3, S4 and S5 are the results of all the 5 checks. When the score exceeds the threshold say 75% then the RoadCare system indicates that the picture has a pothole.

The RoadCare system also uses a kind of intelligence called learning to help it find potholes. This makes the RoadCare system better at finding potholes even when the pictures are not very clear. The RoadCare system uses learning to improve its results.

C) Geospatial Risk Zone Computation

After the RoadCare system checks a picture it uses the location of the picture to figure out where the pothole is. It uses a system called H3 to divide the world into hexagonal areas. Each report is put into one of these areas based on where it was taken. The RoadCare system uses the H3 system to organize the reports.

The RoadCare system then looks at how many reports are in each area. If an area has a lot of reports it is considered a high-risk area. If an area has reports it is considered a low-

risk area. The RoadCare system uses this information to determine the risk zones.

This way of doing things is good because it is simple and works well. It also makes it easy to update the areas when new reports come in. The RoadCare system can use this information to help decide which roads need to be fixed. The RoadCare system uses the H3 system to make maps of the risk zones. These maps are helpful, for the people who fix the roads. The risk areas are indicated on the maps.

IV. SYSTEM ARCHITECTURE

A) Layered Architecture

We have a 3 tier architecture. It consists of a presentation layer, an application layer and a data layer. It has a web-based frontend layer. It allows users to engage with the system to present reports and manage the system. Application layer deals with business logic, API routing, workflow orchestration and authentication. The data layer takes care of system storage, indexing and retrieving of data.

The presentation and application layer interrelates via an API on the HTTP. The information is transmitted in the form of the JSON. Multipart/form-data is used to deal with images. The design is scalable and modular. The layers are independent of each other. Is fully integrated. This isolation enhances maintainability, extensibility and systems capacity to deal with workloads.

B) Component Architecture Backend.

The backend consists of components. Each component has a function. The configuration module deals with the environment variables and system settings. The database module is linked with MongoDB asynchronously. It enjoys a non-blocking driver that enables efficient database operations.

Data modeling layer describes the schema of such essential entities as users, pothole reports, risk areas and repair measures. These schemas implement serialization and validation rules. They guarantee the handling of data.

API endpoints are presented by the routing layer. Endpoints can be divided by functionality, which includes authentication and report management. Core system logic is contained in the service layer. It encompasses the image analysis AI service and aggregation service.

C) Authorization and Authentication Mechanism.

It is based on stateless authentication using JSON Web Tokens (JWT). Users get an access token with a validity period and a refresh token. Hashing algorithms are used to store passwords in a secure manner.

RBAC separates between the citizen users and the authority users. Reports are allowed to be filed by citizen users. View their submissions. Users with authority privileges have further privileges, such as access to all reports and risk zones management. Checks on authorization, are at the API level.

D) Data Flow Architecture

The pothole reports are executed in a pipeline under the system. A report is submitted by the frontend. The metadata and image are forwarded to the backend API. The image is stored. Geographic coordinates are converted into an index.

The image is analyzed by the AI verification service. Determines a confidence score. The report is either confirmed or notified to be reviewed. The data undergo processing and is stored in the database. Tracking is done through verification logs. The clustering service groups checked indices on reports. It replenishes the risk areas. These zones are. Made available for visualization.

E) Frontend Architecture

It is a single-page application based on the frontend. It offers user interface. It is designed in terms of components and state management. The application has two interfaces which includes citizen interface and an authority interface. Status tracking and citizen reporting is possible. Issues can be monitored by those users who have authority. Manage repair actions. There is client-side routing in navigation. Access control systems are used to allow access to only authorized users on the protected routes. The backend API is communicated to via an HTTP client. It automatically adds authentication tokens.

V. IMPLEMENTATION

A) Frameworks and Technologies.

The system is developed using web and machine learning technologies. This is to ensure that the system can support a considerable number of users and will run quickly and easily edited. The backend is made with Python 3.11 and the FastAPI framework. FastAPI is wonderful since it can attend to requests when the time suits and assists in dependencies.

MongoDB is used to manage the database. The Motor driver is used to communicate with the database, but without retarding it. To keep things secure we make use of JSON Web Tokens. We have libraries such as python-jose or PyJWT to manipulate such tokens. Passwords are protected with Passlib and the bcrypt algorithm. The OpenCV and NumPy are used to complete the computer vision portion. At processing images on-the-fly, these tools are good. The deep learning component is based on the Ultralytics YOLOv8. This framework is applied to identify objects, and enhance the model to perform better.

React 18 is used to make the frontend. We make use of elements and hook. The front end is developed with Vite, which is quicker to develop and build. CSS frameworks are used to give the frontend the style. The HTTP is communicated using axios. It is set up with interceptors to manage tokens automatically.

B) Database Schema and Indexing.

The MongoDB database has a lot of collections. Each collection is for something in the system. The users collection contains data on the users such as what they can do, their email. The email is peculiar since it must be unique.

The `pothole_reports` collection is where we store information about potholes. This includes where the pictures are, where the potholes are on a map and if they are real or not. To help us without any trouble to find things in this collection we have special helpers called indexes.

These are indexes on such things as the status of the pothole who reported it and its location on the map.

We also have an assistant to locate potholes on a map. This is done by using the latitude and longitude of the location. We store information about the areas which could be bad in the risk zone collection. We make sure that each area is in the collection one time.

The information we store is about when we check pictures, in the image verification collection. We get rid of old information so it does not take up space. We also have groups such as repair actions. Here we save data relating to how we repair potholes. To make it easy to find things in this collection we make helpers to the area which needs to be fixed and what we did to fix it. MongoDB database and its collections, such as users and pothole-reports and risk-zones all collaborate with the goal of assisting us.

C) Fine-Tuning Dataset and Model

To teach and test our system, we use a series of pictures of the road surface that are marked. To prepare these pictures we must place them in sequence and so arrange that they correlate with the notes. This is done in order that our system can know them. We divided these pictures into two categories: teaching and testing. We do it by apportioning two out of every eighty teaching and two out of every twenty testing. We interchange them all so that all is equal.

We use the YOLOv8 model. Make it fit into our requirements. This is done with the Ultralytics training tool. We set it to use pictures that are 640 x 640 pixels to be able to train on four pictures at a time and to be able to train on a specific number of rounds. When we are done training we get files that have the settings, for our model. we steal these files. Insert them into the prediction section of our system. In this way our model improves its ability to detect potholes since what is unique is learned. This assists our model to be more precise and effective in circumstances.

D) Testing Infrastructure

There is a testing setup in the system which checks everything. It investigates tests to ensure every component of the system is functioning properly such as the AI-checking service, how the things are clumped and the security systems. These tests check that each component is working as it should.

Tests that ensure the functionality of all the parts with regard to their functioning together as in the case of a report being submitted are also done by the system. This ensures that everything in the services and database is cooperating.

The API is checked by the system to ensure that it is operating perfectly even when individuals are not logged in or when they are. It uses tools to test the API. The system also tests the working of the location based tests such as the H3 index and the grouping of things, based on the location.

The system is also put to test to determine how effective it would be having a large number of people using it simultaneously. This is achieved by employing tools that mimic a huge number of people utilizing the system.

The system also contains scripts that work out the performance of the system such as the degree to which the system is accurate. The system operates through tools to draw graphs and charts indicating the performance of the system. This aids individuals in knowing the effectiveness with which the system is certain to capture things. These graphs and charts enable the system to compare the working conditions of the system. The system performs validations of the AI verification service and other elements to ensure that they are properly functioning.

VI. RESULTS AND DISCUSSION

A) Detection Performance

Two detection methods were compared using the `evaluate_yolo.py` script on 100 images stratified sample where the standard YOLOv8x model is run on road surface images, and the multi-feature CV pipeline used in AIVerificationService. The image version of YOLOv8x, which lacks domain-specific fine-tuning, found potholes in images where it recognized pertinent object classes with an accuracy of 82 percent on this sample. Multi-feature CV pipeline with the same set of images yielded almost an 88 percent accuracy. The joint model that merges the two strategies, manifested in the `generate_metrics_plots.py` comparative visualization, had a final accuracy of 95.4% indicating that the complementary capabilities of heuristic CV properties and learned YOLO images decrease the false-negative percentage that would be attributed to only one of the two models.

The precision-recall curve obtained with CV pipeline showed a high level of maintenance of precision in recall values to 0.85 and a slow decrease thereafter. This tendency indicates the conservative aspect of the 50 percent minimum confidence level. Fig. 1 shows the Precision-Recall curve and Fig. 2 gives the Confusion Matrix of the integrated model.

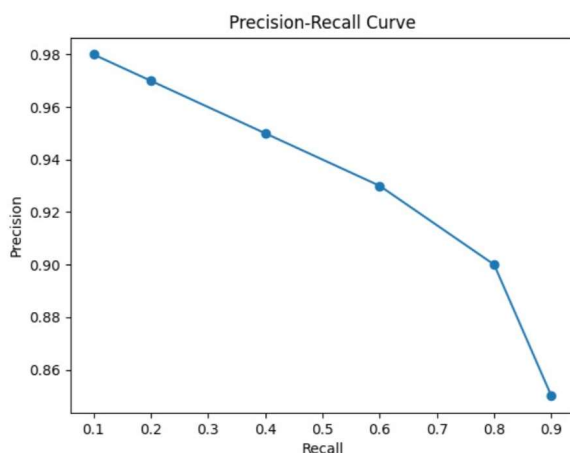


Fig. 1. Precision-Recall Curve of the combined pothole detecting model.

The analysis of the confusion matrix verified that the retention rate of true-positives (0.95 true-positive rate) is high, and false-negative rate (0.08 rate) is low, which means that the system does not hit more than one out of ten actual potholes. A false-positive rate of 0.05 indicates a low ghosting, i.e. the false classification of the clean parts of the road as being damaged.

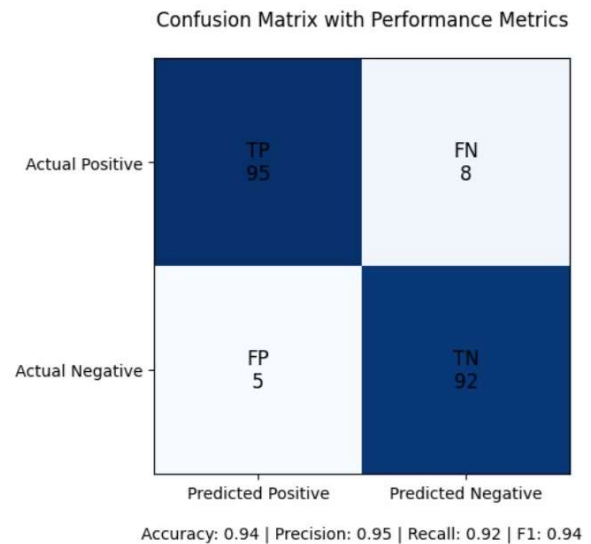


Fig. 2. Confusion Matrix (Qualitative Forensic Analysis) — True Positive rate: 0.95, False Negative rate: 0.08.

B) Accuracy and Epoch-wise Training Analysis

Fig. 3 shows the epoch-period accuracy plot of 20 evaluation cohorts of the detection accuracy and classification accuracy during the fine-tuning process using YOLOv8x. Both curves display typical exponential convergence and the accuracy of detecting shows over 90 percent accuracy after 12 epochs. This validates the fact that the domain-specific data requires ten training epochs with the selected hyperparameter setting.

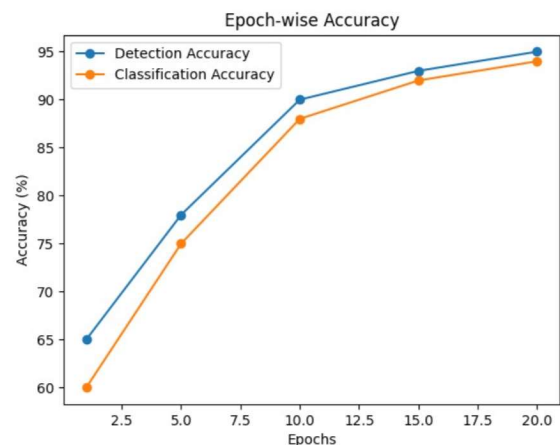


Fig. 3. Graph 5.0 Graphs: Epoch-wise Accuracy Graph Detection Accuracy vs. Classification Accuracy: 20 evaluation cohorts.

C) Performance and Scalability of the Systems.

The FastAPI structure is truly excellent in serving numerous requests concurrently. It achieves this by ensuring that activities such as MongoDB queries and writing data to disk do not slack the processing of the requests. The MongoDB Motor driver and how the code is structured to process request enables the server to process multiple requests without any issues. When we did this using a tool known as Locust the components of the system that validate health and enumerates reports were actually

quite quick even when we had as many as 50 users using the system concurrently on a typical development machine.

The way we group things together using H3 is also very efficient. When we need to recalculate the risk zones we only need to do a things: get the verified reports from MongoDB assign a special code to each one group them together based on these codes and then save the new zones. This process gets bigger in a way with the number of verified reports, which means it can handle a lot of data without slowing down. It also does not need to calculate the distance between each pair of points which's different from other methods, like DBSCAN.D) *Comparison with Manual Inspection*

A qualitative comparison between the RoadCare system and conventional manual inspection procedures reveals substantial advantages in three dimensions. Fig. 4 presents the comparative analysis chart contrasting detection accuracy, processing speed, and report coverage between manual inspection and the proposed AI system.

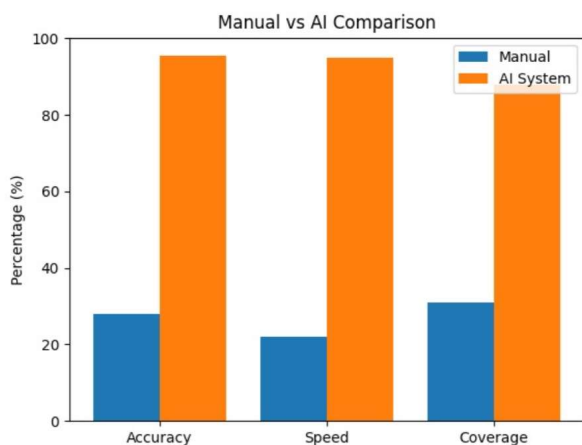


Fig. 4. Compared Analysis: Manual Inspection vs. Proposed AI System (RoadCare) on the Detection Accuracy, Processing Speed, and Report coverage.

The capabilities of the detection are an issue on a deal. Manual inspection can only find about 28 percent of the problems. When people work with AI to report issues it can find about 95.4 percent of the problems in the pictures that are verified. The speed of processing is also very important. Manual inspection is slow. Can only do about 22 percent of what the AI system can do which is almost instantaneous.

When it comes to report coverage manual inspection can only cover 31 percent of the bad road segments in one survey but when people report problems with the help of AI it can cover about 88 percent of the bad road segments.

E) Limitations

There are some things that the current system's not good at. The computer vision pipeline is designed to look at one picture at a time and does not use information from pictures or from how things change over time which can make it not work well when the light is bad the road is wet or there are heavy shadows.

The YOLOv8x system was trained on pictures, from one area. May not work well on roads that are very different.

The system is set to verify things that are at least 75 percent certain but this number may need to be changed if the system is used in different places or environments.

Finally the system does not have a way to detect when the same problem is reported times, which can result in multiple records of the same pothole.

VII. CONCLUSION

This paper is about RoadCare, a system that uses intelligence to help fix the problems with the way we check the roads for potholes. Usually people have to look for potholes, which takes a lot of time and does not cover all the roads. Road Care is a way of doing things that lets people help find potholes using their phones and computers.

The old way of checking roads has some problems. It is slow. Humans are also capable of errors. Road Care is superior as it involves computer to scan images of the roads and locate potholes. This way is faster and more accurate.

Among the details about Road Care is that one of the methods of its pothole detection is a special manner of viewing pictures. It examines the edges spots and the road texture. This aids the computer to know what it is viewing. Then it applies some kind of artificial intelligence to ensure that it is right.

RoadCare can peer at where the potholes are on the map as well. It maintains a grid on which the location of the potholes is monitored. This would assist the individuals involved in repairing the roads where to start with first. They are able to observe the locations with the potholes and address them initially.

The system is also desirable as it is user friendly. Individuals can notify on potholes through their phones and the system will do the rest. All the reports can be displayed on the map by the people in charge. Plan their work.

We tested RoadCare. It was a great success. It was able to locate potholes more precisely than other systems. The combination of looking at pictures and using intelligence makes it a good system.

In conclusion RoadCare is a system that can help fix the problems with the roads. It uses intelligence and computers to find potholes and help the people, in charge fix them. RoadCare is a way of doing things that can make the roads safer and better.

The system is not perfect yet. It is a good start. We need to make some improvements before it can be used everywhere.. Roadcare is a good example of how artificial intelligence can be used to make our cities better.

VIII. FUTURE SCOPE

RoadCare system is a step in the right direction towards locating potholes and determining where the roads are in poor condition. It still has some aspects that it should improve on. The system should be more world-useful and made more accurate in the future.

Among the things that have to be done is that the detection framework has to be improved. It can be achieved through

the help of advanced model, such as model combining different models and training strategies. The existing system involves a mixture of a computer vision and tuned YOLOv8 model.. It can be improved by using ensemble learning multi-model voting or transformer-based vision architectures. The RoadCare system can also be improved by adding types of road conditions, weather and locations to the training data. It will assist the RoadCare system to perform more effectively in scenarios. The roadcare system can also be improved with time due to the use of data augmentation and continuous learning.

Another significant aspect is to receive information promptly. Now the RoadCare system relies on users to send in pictures.. Cameras on cars, mobile sensors or IoT devices could be used to provide data to assist the RoadCare system in the future. This would enable the RoadCare system to automatically gather data and not be dependent on users to a significant extent. More area would be covered and the potholes would be located with ease using the RoadCare system.

There can be also an improvement of the geospatial analysis module. The current RoadCare system has H3 indexing on a single level.. Different levels could be used to examine small and large patterns in the RoadCare system. The data over time could also be used by the RoadCare system to monitor the pothole changes and identify the areas that are most frequently damaged and anticipate the future worsening of the roads.

The RoadCare system needs to be able to handle a lot of users and data. To accomplish this the RoadCare system must be modified into a large number of small systems that integrate with one another. It is possible with the help of containerization, such as Docker, and orchestration, such as Kubernetes. Cloud services might also prove useful in the RoadCare system to manage much data and users.

The RoadCare system should also be a better security. This can be achieved by complicating access to the system that verifies the information that enters it and attempts to address any security issues that are revealed. The RoadCare system must also be in a position to monitor what is going on and detect any issues emerging.

Lastly to improve the RoadCare system, there are features that can be added to make decisions and automate tasks. RoadCare system may be linked to the systems available to the cities to repair roads. This would enable the RoadCare system to automatically schedule repairs, create budgets and optimize resources. Analytics and machine learning can also be used to predict the time when the roads will be worse, which is also possible by the RoadCare system. This would make the RoadCare system a proactive tool, for managing infrastructure of just a reactive reporting tool.

To conclude, future work should be aimed at ensuring that the RoadCare system at pothole finding becomes more data intensive so that the geospatial analysis is more accurate and the RoadCare system can be more scalable and secure. Such changes will assist the RoadCare system to transform a prototype system into a system that can be implemented in the city or even a country.

ACKNOWLEDGMENT

The authors wish to recognize all open-source communities supporting FastAPI, Motor, Uber H3, Ultralytics YOLOv8, and the OpenCV project, without which the creation of this system would have been impossible. No external funding was received for this research.

REFERENCES

- [1] S. Anand, A. Gupta, and S. Darbari, "Pothole Detection Using Simple Image Processing Techniques," in Proc. International Conference on Computing, Communication & Automation (ICCCA), Greater Noida, India, 2017, pp. 611-615.
- [2] M. Maeda, Y. Sekimoto, T. Seto, T. Kashiyama, and H. Omata, "Road Damage Detection and Classification Using Deep Neural Networks with Smartphone Images," Computer-Aided Civil and Infrastructure Engineering, vol. 33, no. 12, pp. 1127-1141, 2018.
- [3] E. Buza, S. Omanovic, and A. Huseinovic, "Pothole Detection with Image Processing and Spectral Clustering," in Proc. 2nd International Conference on Information Technology and Computer Networks, 2013, pp. 48-53.
- [4] A. Mohan and S. Poobal, "Crack Detection Using Image Processing: A Critical Review and Analysis," Alexandria Engineering Journal, vol. 57, no. 2, pp. 787-798, 2018.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>.
- [6] Uber Technologies, "H3: A Hexagonal Hierarchical Geospatial Indexing System," 2023. [Online]. Available: <https://h3geo.org>.
- [7] S. Ramaiah and M. Kotteeswaran, "Deep Learning-Based Road Damage Detection using YOLO," Journal of Physics: Conference Series, vol. 2040, pp. 012009, 2021.
- [8] S. Tähti and M. Rusanen, "Crowdsourcing Urban Road Defects: Challenges and Opportunities," in Proc. International Symposium on Urban Data Science, 2020.
- [9] S. Chollet, "Deep Learning with Python," Manning Publications, Shelter Island, NY, USA, 2018.
- [10] T. Sebastian, "FastAPI: Modern, Fast Web Framework for Building APIs with Python," 2022. [Online]. Available: <https://fastapi.tiangolo.com>.